PartZone

장바구니 백엔드 구현 가이드

React + Vite / Spring Boot + JPA / Oracle / JWT 현재 구조에 맞춘 설계와 작성 순서

> 이 문서는 현재 프로젝트의 USERS, PARTS, JWT 인증, GlobalExceptionHandler 구조를 기준으로 작성했다. 블로그 예제의 Thymeleaf, MariaDB, Principal 기반 코드는 개념만 참고하고 그대로 복사하지 않는다.

완성 목표

로그인 사용자가 부품을 장바구니에 담고, 수량을 변경하거나 삭제하며, React 화면에서 선택한 상품만 주문 페이지로 전달할 수 있게 만든다.

단계	결과
1	Oracle에 CART, CART_ITEM 테이블과 Sequence 생성
2	cart 도메인의 Entity, Repository, DTO 작성
3	JWT 사용자 기준 Service와 Controller 구현
4	React CartContext를 API 호출 방식으로 전환
5	선택한 cartItemId만 주문 요청으로 전달

먼저 기억할 경계

장바구니는 저장 기능이고, 주문은 결제 직전의 확정 기록이다. 따라서 CART/CART_ITEM과 ORDER/ORDER_ITEM은 나중에도 합치지 않는다.

1. 현재 프로젝트에 맞는 데이터 모델

관계는 아래처럼 잡는다. 사용자는 장바구니 하나를 가지며, 장바구니 안에는 여러 부품 행이 들어간다.

```
USERS (1) ---- (1) CART (1) ---- (N) CART_ITEM (N) ---- (1) PARTS
```

테이블	핵심 컬럼	역할
CART	cart_id, user_id	사용자 한 명의 장바구니 컨테이너
CART_ITEM	cart_item_id, cart_id, part_id, quantity	장바구니의 부품 한 줄
PARTS	part_id, price	현재 판매 가격을 가진 기존 부품 테이블
USERS	user_id, email	JWT로 식별되는 기존 사용자 테이블

왜 CART와 CART_ITEM을 나누는가?

한 사용자가 CPU, GPU, RAM을 동시에 담을 수 있기 때문이다. CART_ITEM 한 행은 '어떤 장바구니에 어떤 부품을 몇 개 담았는가'를 표현한다.

저장하지 않는 값

값	이유
총금액	PARTS.price x CART_ITEM.quantity로 계산한다. 별도 저장 시 수량·가격 변경 후 불일치 위험이 있다.
선택 체크 여부	주문 직전의 화면 상태다. selectedIds는 React/주문 요청에서만 사용한다.
장바구니 상품 수	CART_ITEM을 조회해 계산할 수 있는 중복 데이터다.

주의

현재 PARTS의 가격은 크롤링 데이터다. 주문을 실제로 확정하는 단계에서는 '현재 가격'을 쓸지 '주문 당시 가격'을 ORDER_ITEM에 복사할지를 별도로 결정해야 한다. 장바구니 단계에서는 현재 PARTS.price를 사용하면 된다.

2. Oracle DDL

현재 테이블은 따옴표를 사용한 소문자 컬럼명으로 매핑하고 있으므로, 새 테이블도 같은 방식으로 통일한다.

```
CREATE SEQUENCE SEQ_CART
START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;

CREATE SEQUENCE SEQ_CART_ITEM
START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;

CREATE TABLE CART (
"cart_id" NUMBER PRIMARY KEY,
"user_id" NUMBER NOT NULL UNIQUE,
"created_at" TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
CONSTRAINT FK_CART_USER
FOREIGN KEY ("user_id") REFERENCES USERS("user_id")
);

CREATE TABLE CART_ITEM (
"cart_item_id" NUMBER PRIMARY KEY,
"cart_id" NUMBER NOT NULL,
"part_id" NUMBER NOT NULL,
"quantity" NUMBER NOT NULL,
CONSTRAINT CK_CART_ITEM_QUANTITY CHECK ("quantity" >= 1),
CONSTRAINT UQ_CART_ITEM_CART_PART UNIQUE ("cart_id", "part_id"),
CONSTRAINT FK_CART_ITEM_CART
FOREIGN KEY ("cart_id") REFERENCES CART("cart_id"),
CONSTRAINT FK_CART_ITEM_PART
FOREIGN KEY ("part_id") REFERENCES PARTS("part_id")
);
```

핵심 제약 조건

제약	막는 문제
CART.user_id UNIQUE	사용자 한 명에게 장바구니가 여러 개 생성되는 문제
CART_ITEM(cart_id, part_id) UNIQUE	같은 부품이 장바구니에 중복 행으로 들어가는 문제
quantity CHECK >= 1	0개, 음수 수량이 DB에 저장되는 문제

DB 제약 조건은 Service 검증을 대체하는 것이 아니라 마지막 안전망이다. 동시에 두 요청이 들어오는 race condition은 Java if문만으로 막을 수 없으므로 UNIQUE가 필요하다.

3. 패키지와 Entity 설계

기존 도메인 구조를 유지한다. 새 폴더는 domain/cart 아래에 만든다.

```
domain/cart/
controller/CartController.java
service/CartService.java
repository/CartRepository.java
repository/CartItemRepository.java
entity/Cart.java
entity/CartItem.java
dto/CartItemAddRequest.java
dto/CartItemQuantityRequest.java
dto/CartItemResponse.java
```

Cart Entity의 핵심

```
@Entity
@Table(name = "CART")
public class Cart {
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "cartSequence")
@SequenceGenerator(name = "cartSequence", sequenceName = "SEQ_CART", allocationSize = 1)
@Column(name = "₩"cart_id₩"")
private Long cartId;

@OneToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "₩"user_id₩"", nullable = false, unique = true)
private User user;

protected Cart() {}

public Cart(User user) { this.user = user; }
}
```

CartItem Entity의 핵심

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "₩"cart_id₩"", nullable = false)
private Cart cart;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "₩"part_id₩"", nullable = false)
private Part part;

@Column(name = "₩"quantity₩"", nullable = false)
private int quantity;

public void addQuantity(int amount) {
this.quantity += amount;
}

public void changeQuantity(int quantity) {
if (quantity < 1) throw new IllegalArgumentException();
this.quantity = quantity;
```

```
}
```

Entity에는 전체 Setter를 열지 않는다. 수량 변경처럼 허용한 행동만 메서드로 만들면 Service가 의도하지 않은 상태를 만들기 어려워진다.

4. API와 Service 흐름

HTTP	경로	역할
GET	/api/cart	로그인 사용자의 장바구니 조회
POST	/api/cart/items	부품 담기: partId, quantity
PATCH	/api/cart/items/{cartItemId}	수량을 지정한 값으로 변경
DELETE	/api/cart/items/{cartItemId}	한 상품 삭제
DELETE	/api/cart	장바구니 전체 비우기

POST 요청 DTO

```
public record CartItemAddRequest(
@NotNull(message = "부품 ID가 필요합니다.") Long partId,
@NotNull @Min(value = 1, message = "수량은 1 이상이어야 합니다.") Integer quantity
) {}
```

Service의 addItem 실행 순서

순서	검증 또는 처리
1	JWT SecurityContext에서 로그인 이메일을 가져온다.
2	UserRepository.findByEmail(email)으로 사용자 확인 - 없으면 401/404.
3	PartRepository.findById(partId)로 실제 부품 확인 - 없으면 404.
4	Cart가 없으면 new Cart(user) 후 저장한다.
5	같은 cartId + partId 행이 있으면 수량 증가, 없으면 CartItem 생성.
6	CartItemResponse로 변환해 JSON 반환.

보안 원칙

요청 JSON에 userId를 받지 않는다. 서버가 JWT의 인증 객체에서 로그인 사용자를 식별한다. 그렇지 않으면 userId만 바꿔 다른 사람 장바구니를 읽거나 수정하는 IDOR 취약점이 생긴다.

현재 SecurityConfig는 `/api/parts/**`만 공개이고 나머지는 인증이 필요하다. 따라서 `/api/cart/**`는 permitAll에 추가하지 않으면 자동으로 JWT 인증이 적용된다.

5. React와 연결하는 순서

현재 CartContext는 브라우저 메모리 상태다. 새로고침하면 사라지는 이유는 서버 API나 localStorage에 저장하지 않기 때문이다. 백엔드 장바구니를 만든 후에는 아래 흐름으로 전환한다.

```
부품 상세의 '장바구니 담기'
-> POST /api/cart/items
-> 서버가 CART_ITEM 저장

장바구니 페이지 진입
-> GET /api/cart
-> 응답으로 CartContext.items 채움

수량 변경/삭제
-> PATCH 또는 DELETE API 호출
-> 성공한 뒤 Context 갱신 또는 GET 재조회
```

체크박스와 주문의 구분

상태	보관 위치	이유
items	DB CART_ITEM + CartContext	장바구니 자체 데이터이며 새로고침 뒤에도 유지되어야 함
selectedIds	CartContext 또는 주문 페이지 이동 state	이번 주문에 포함할 항목만 고르는 일시적 화면 상태
selectedTotalPrice	items와 selectedIds에서 계산	별도 저장하면 수량 변경 시 금액이 틀어질 수 있음

주문 페이지로 넘길 데이터

```
POST /api/orders
{
"cartItemIds": [12, 19, 24]
}
```

주문 API가 받은 cartItemIds는 반드시 '현재 로그인 사용자 소유의 CART_ITEM인가'를 서버에서 다시 확인해야 한다. 프론트가 보낸 가격·상품명·합계는 신뢰하지 않고, 서버에서 PARTS와 CART_ITEM을 다시 조회해 계산한다.

구현 체크리스트

확인	완료 기준
인증	토큰 없이 /api/cart 호출 시 공통 401 JSON 응답
중복 담기	같은 partId를 두 번 담으면 행 증가 대신 quantity 증가
삭제	다른 사용자의 cartItemId를 삭제하려 하면 403 또는 404
새로고침	GET /api/cart로 DB 저장 장바구니가 다시 표시
주문	체크한 cartItemId만 주문 요청에 포함